Computer Vision Homework 8

Name: NICKHOLAS

Student ID: M11515801

Department: NTUST CSIE

Write a program to do the following requirement.

I am using python 3.10 to implement the noise removal requirement.

Using OpenCV (cv2) only to read and write the image and to pad the border of the image, and all of the filters are hand-written on NumPy arrays. All of the results are saved as .bmp files.

1. Results

Gaussian noise with amplitude of 10



gaussian noise with amplitude 10, snr=13.5836



3x3 box filter
snr=17.9782



3x3 median filter
snr=17.8553



opening-then-closing
snr=13.2732



5x5 box filter
snr=14.9556



5x5 median filter
snr=16.0891



closing-then-opening
snr=13.6189

Gaussian noise with amplitude of 30



gaussian noise with amplitude 30, snr=4.1601



3x3 box filter
snr=12.6333



3x3 median filter
snr=11.0941



opening-then-closing
snr=11.1674



5x5 box filter
snr=13.3347



5x5 median filter
snr=12.9272



closing-then-opening
snr=11.2484

Salt-and-Pepper noise with probability 0.1



salt-and-pepper noise with probability 0.1, snr=−2.1093



3x3 box filter
snr=6.3320



3x3 median filter
snr=15.0325



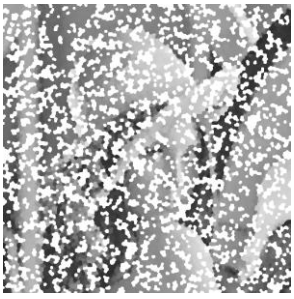
opening-then-closing
snr=−2.1336



5x5 box filter
snr=8.5168



5x5 median filter
snr=15.9033



closing-then-opening
snr=−2.6058

Salt-and-Pepper noise with probability 0.05



salt-and-pepper noise with probability 0.05, snr=0.9280



3x3 box filter
snr=9.5487



3x3 median filter
snr=19.5504



opening-then-closing
snr=5.8644



5x5 box filter
snr=11.2390



5x5 median filter
snr=16.4889



closing-then-opening
snr=5.2498

2. Code fragment

```python
def gaussian_noise(img, amplitude, rng):
    noisy = img.astype(np.float64) + amplitude * rng.normal(0.0, 1.0, img.shape)
    return np.clip(np.rint(noisy), 0, 255).astype(np.uint8)


def salt_and_pepper_noise(img, prob, rng):
    u = rng.uniform(0.0, 1.0, img.shape)
    noisy = img.copy()
    noisy[u <= prob] = 0
    noisy[u >= 1.0 - prob] = 255
    return noisy
```

```python
def box_filter(img, k):
    mean = np.sum(neighborhood(img, k), axis=0, dtype=np.float64) / (k * k)
    return np.clip(np.rint(mean), 0, 255).astype(np.uint8)


def median_filter(img, k):
    return np.sort(neighborhood(img, k), axis=0)[(k * k) // 2]
```

```python
OCTAGON = [(dy, dx)
           for dy, half in [(-2, 1), (-1, 2), (0, 2), (1, 2), (2, 1)]
           for dx in range(-half, half + 1)]

def shift(img, dy, dx, fill):
    h, w = img.shape
    out = np.full_like(img, fill)
    ys, ye = max(0, -dy), min(h, h - dy)
    xs, xe = max(0, -dx), min(w, w - dx)
    out[ys:ye, xs:xe] = img[ys + dy:ye + dy, xs + dx:xe + dx]
    return out


def dilation(img, kernel):
    out = np.zeros_like(img)
    for dy, dx in kernel:
        out = np.maximum(out, shift(img, -dy, -dx, 0))
    return out


def erosion(img, kernel):
    out = np.full_like(img, 255)
    for dy, dx in kernel:
        out = np.minimum(out, shift(img, dy, dx, 255))
    return out
```

```python
def opening(img, kernel):
    return dilation(erosion(img, kernel), kernel)


def closing(img, kernel):
    return erosion(dilation(img, kernel), kernel)
```

```python
def opening_then_closing(img):
    return closing(opening(img, OCTAGON), OCTAGON)


def closing_then_opening(img):
    return opening(closing(img, OCTAGON), OCTAGON)
```

```python
def snr(original, processed):
    a = original.astype(np.float64) / 255.0
    b = processed.astype(np.float64) / 255.0
    n = a.size
    mu = np.sum(a) / n
    vs = math.sqrt(np.sum((a - mu) ** 2) / n)
    mu_n = np.sum(b - a) / n
    vn = math.sqrt(np.sum((b - mu_n - a) ** 2) / n)
    return 20.0 * math.log10(vs / vn)
```

3. Brief Description

All the assigned operators' implementation details follow the course's lecture slides. The np.random was used to simulate all the assigned noise distribution. The gaussian noise is I' = I + amplitude x N(0, 1), and the result is rounded and clipped to 0~255. For the salt-and-pepper noise, a uniform random number u is generated for each pixel: the pixel is set to 0 if u <= probability, set to 255 if u >= 1 - probability, and otherwise unchanged. A fixed random seed is used so that the result can be reproduced.

For the box filter and the median filter, the image is padded by reflection first, so that the output has the same size as the original image. For each position in the k x k window, a shifted copy of the padded image is made, and these copies are stacked. The box filter takes the mean of the stack, and the median filter sorts the stack and takes the middle value.

The opening and closing operations use the octagonal 3-5-5-5-3 kernel with value = 0, and the dilation and erosion are reused from HW4 (maximum and minimum over the kernel, the outside of the image is padded with 0 for dilation and 255 for erosion). Opening-then-closing is closing(opening(I)) and closing-then-opening is opening(closing(I)). Please note that the boundary pixel's values of the opening and closing outputs depend on the padding, so that the SNR values may be a little different from the reference.

The images are normalized from 0~255 to 0~1 before calculating the SNR. SNR = 20 x log10(VS / VN), where VS is the standard deviation of the original image, and VN is the standard deviation of the noise I' - I. The SNR is calculated for the 4 noisy images and the 24 processed images.